



**Tecnológico
de Monterrey**

TC4031.10

Cómputo en la nube (Gpo 10)

Tarea 7. Usar APIs en la nube

Jaime Alejandro Mendivil A. A01253316

15 de Marzo del 2026

Introducción

En esta práctica se explorará el uso de APIs proporcionadas por dos de los principales proveedores de nube: Google Cloud Platform y Microsoft Azure. A través de scripts en Python se consumirá el servicio de Cloud Translation para realizar traducciones automáticas y el servicio de Azure Cognitive Services para analizar imágenes y detectar rostros. Además, se investigará e integrará un servicio adicional accesible mediante API, con el objetivo de comprender cómo configurar, consumir y documentar estos servicios dentro de una aplicación.

De esta manera se busca familiarizarse con el uso práctico de APIs en la nube, así como con el proceso de autenticación, configuración de credenciales y consumo de servicios inteligentes que pueden enriquecer el desarrollo de aplicaciones modernas.

Servicio en MS Azure

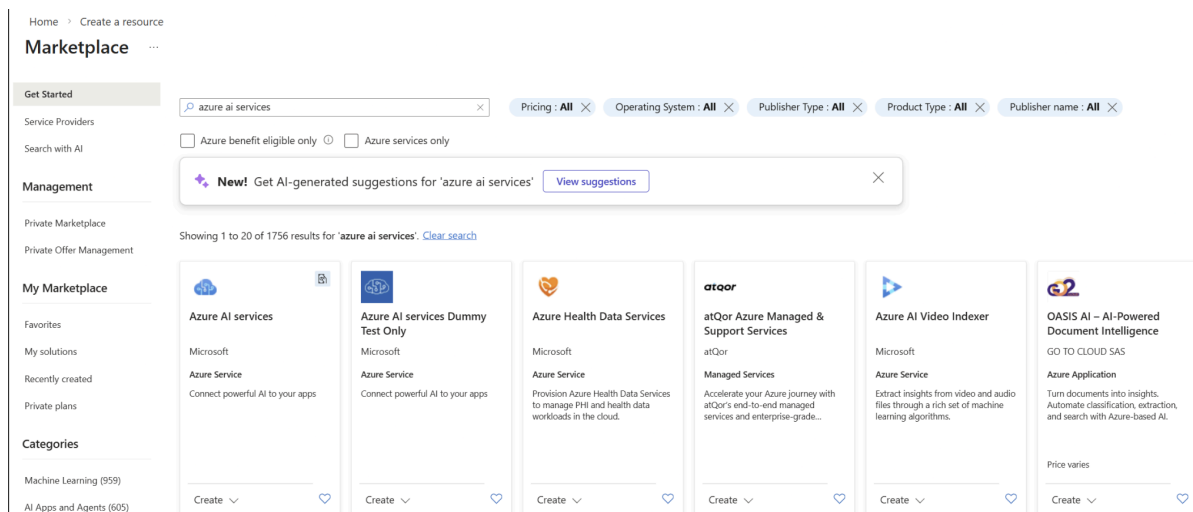


Figura 1. Creación de recurso

Empezaremos creando un recurso y añadiremos “Azure Ai Services”. Para ello necesitaremos saber las regiones permitidas en nuestra póliza para la cuenta de estudiantes.

Parameter value ↑↓

```
["eastus","mexicocentral","chilecentral","canadacentral","westus3"]
```

Figura 2. Regiones permitidas.

Ya con esto creamos el recurso con los siguientes datos.

Home > Create a resource > Marketplace > Azure AI services

Create Azure AI services

Basics Network Identity Tags Review + create

Get access to Vision, Language, Search, and Speech Azure AI services with a single API key. Quickly connect services together to achieve more insights into your content and easily integrate with other services like Azure Search.

[Learn more](#)

Project Details

Subscription *	Azure for Students
Resource group *	(New) GR-FaceRecog-API-Python

[Create new](#)

i Azure AI services resource creation requires subscription registration, we detected that your selected subscription did not register Cognitive services resource type before, we will help you to register Cognitive services resource type when you select a subscription in subscription dropdown. Click to learn more how to check registration state for your selected subscription.

Instance Details

Region	West US 3
Name *	FaceRecognitionMNA01253316

i Location specifies the region only for included regional services. This does not specify a region for included non-regional services. Click here for more details.

Pricing tier *	Standard S0
----------------	-------------

[View full pricing details](#)

Figura 3. Datos del recurso

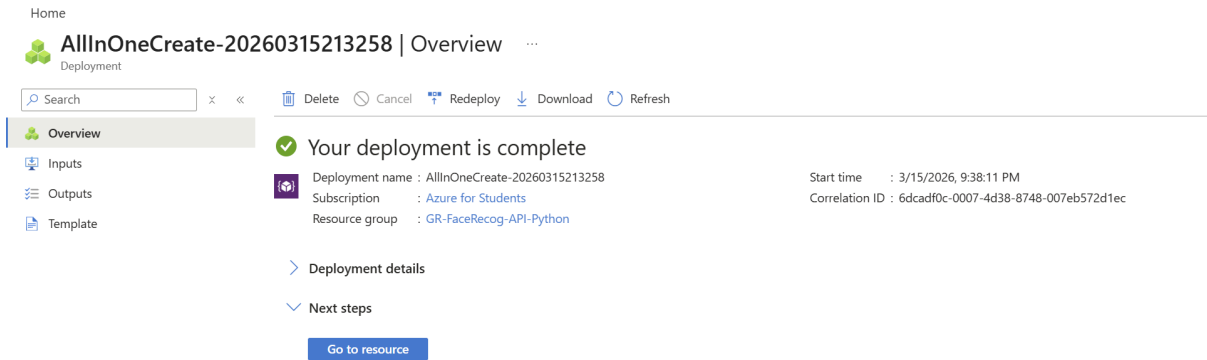


Figura 4. Creación del recurso exitosa

Crear llave para Cognitive Services (CS)

Ahora generamos nuestra llave API

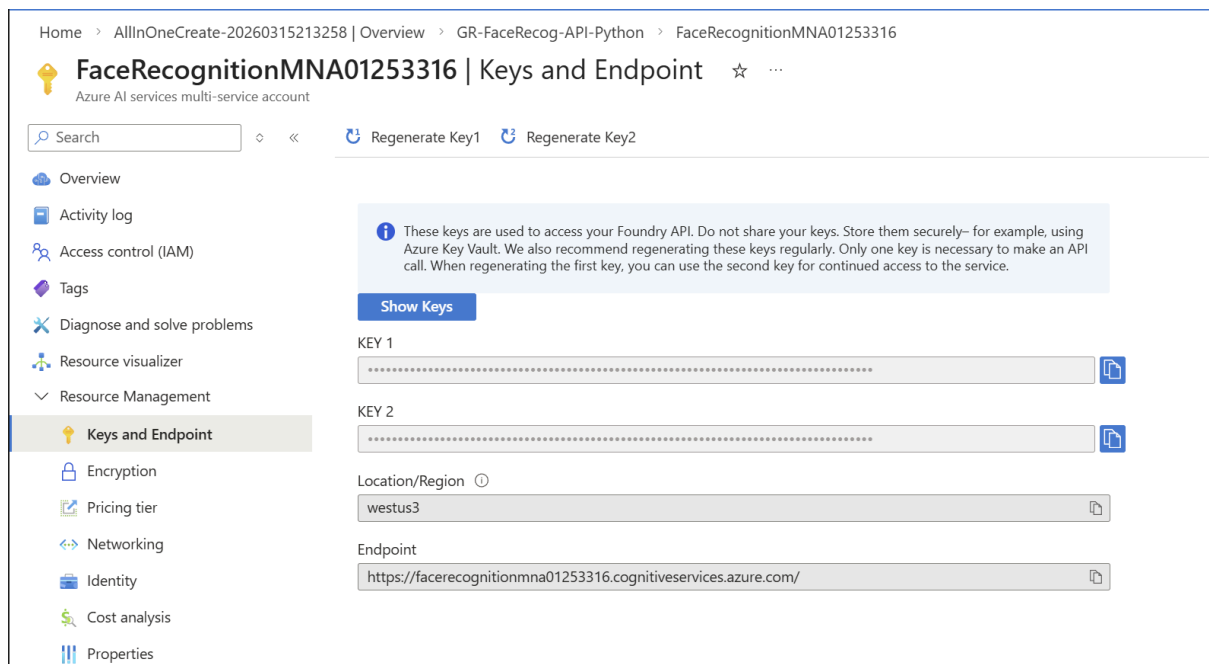


Figura 5. Llaves API

Copiaremos las llaves para ser usadas en nuestro archivo .venv o haciendo uso de un administrador de claves secretas.

Crear código de Python para usar la API CS y comentar las líneas con

Creemos un archivo de Google Collab e insertamos las API keys en la pestaña de secretos.

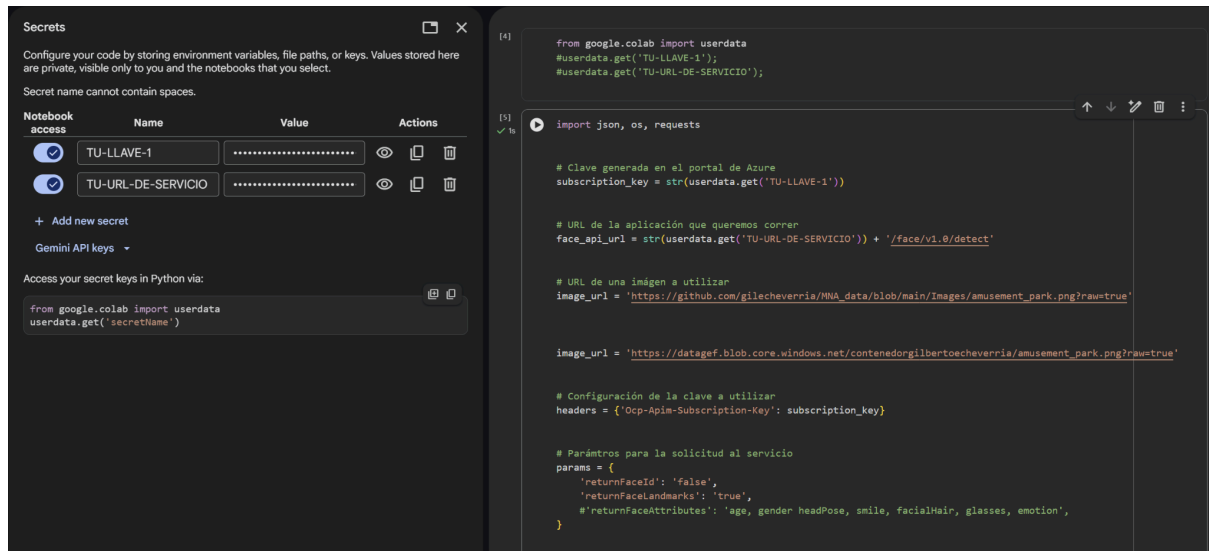


Figura 6. Archivo de Google Collab.

De esta forma no exponemos nuestras claves si son subidas a algún repositorio público. Y con esto checamos la salida.

```
'[{"faceRectangle": {"top": 352, "left": 467, "width": 50, "height": 50},
"faceLandmarks": {"pupilLeft": {"x": 479.8, "y": 366.1}, "pupilRight": {"x":
503.6, "y": 367.2}, "noseTip": {"x": 493.8, "y": 377.7}, "mouthLeft": {"x": 479.6,
"y": 388.1}, "mouthRight": {"x": 500.7, "y": 389.4}, "eyebrowLeftOuter": {"x":
470.8, "y": 361.6}, "eyebrowLeftInner": {"x": 489.6, "y": 361.4}, "eyeLeftOuter":
{"x": 477.3, "y": 366.6}, "eyeLeftTop": {"x": 480.2, "y": 365.6}, "eyeLeftBottom":
{"x": 480.2, "y": 367.2}, "eyeLeftInner": {"x": 483.0, "y": 366.6},
"eyebrowRightInner": {"x": 497.2, "y": 361.6}, "eyebrowRightOuter": {"x": 513.3,
"y": 363.2}, "eyeRightInner": {"x": 500.1, "y": 367.2}, "eyeRightTop": {"x":
503.6, "y": 366.3}, "eyeRightBottom": {"x": 503.7, "y": 367.7}, "eyeRightOuter":
{"x": 506.6, "y": 367.4}, "noseRootLeft": {"x": 488.6, "y": 367.0},
"noseRootRight": {"x": 496.0, "y": 367.1}, "noseLeftAlarTop": {"x": 487.1, "y":
374.2}, "noseRightAlarTop": {"x": 497.6, "y": 374.6}, "noseLeftAlar0...<'
```

Figura 7. Salida JSON del código.

Mostrar la imagen y el área detectada

A continuación agregaremos más código para observar bien la imagen original con los bounding boxes sobrepuestos.

```
from PIL import Image, ImageDraw
from io import BytesIO
import matplotlib.pyplot as plt

# Descargar los datos de la imagen
img_data = requests.get(image_url)
# Crear un objeto para poder dibujar la foto
img = Image.open(BytesIO(img_data.content))

# Nuevo objeto para poder dibujar figuras sobre la foto
draw = ImageDraw.Draw(img)

# Mostrar un rectángulo alrededor de las caras detectadas
for face in faces:
    rect = face['faceRectangle']
    left = rect['left']
    top = rect['top']
    right = left + rect['width']
    bottom = top + rect['height']

    draw.rectangle([left, top, right, bottom], outline='red', width=3)

# Formateo de la imagen resultante
plt.figure(figsize=(14, 10))
plt.imshow(img)
plt.axis('off')
plt.title(f"{len(faces)} face(s) detected")
plt.show()
```

Figura 8. Código para desplegar salida de API sobre la imagen original.

Lo que hace este código es obtener la salida JSON del API de detección de caras y después desplegamos los “bounding boxes” en la imagen si tienen la etiqueta de ‘faceRectangle’

8 face(s) detected



Figura 9. Detección de rostros en imagen de ejemplo.

Obtener los resultados e interpretarlos

Con esto podemos observar todos los pasos del proceso, en donde podemos visualizar que la salida JSON que nos devuelve la API a partir de nuestra imagen son las ubicaciones en donde se detectaron rostros, así como las etiquetas de dicha segmentación. Después de esto procedemos a ubicar en nuestra imagen original las áreas en donde se detecte una bounding box con la etiqueta de 'faceRectangle' para evitar que nuestra imagen se llene con las detecciones de ojos, pestañas, cejas y otras etiquetas adicionales que puedan ofuscar la imagen.

Investigar e implementar otros servicio a través de una API

Investigar el Servicio

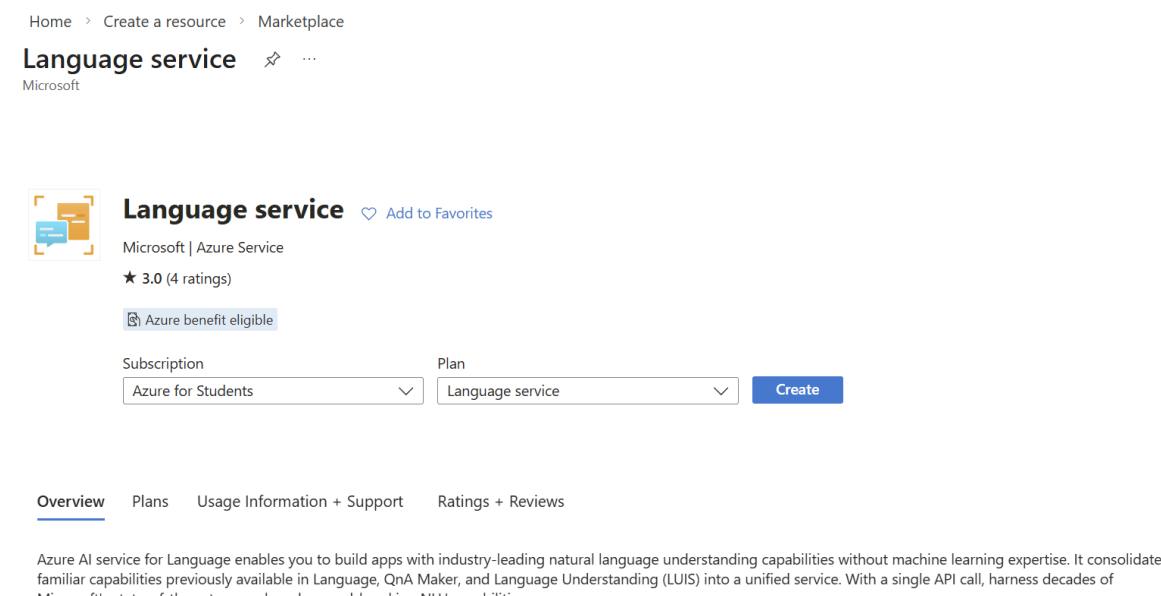
Azure AI Language – Sentiment Analysis es un servicio de inteligencia artificial que forma parte de los servicios cognitivos de Microsoft dentro de la plataforma de computación en la nube Microsoft Azure.

Este servicio utiliza técnicas de procesamiento de lenguaje natural (NLP) para analizar textos y determinar automáticamente la actitud o emoción expresada en ellos.

El análisis de sentimiento permite clasificar un texto en categorías como positivo, negativo o neutral, cosa que es extensamente usada para el área de satisfacción de usuarios en redes sociales, reseñas de productos y opiniones en general.

Configurar el servicio

Empezaremos creando otro recurso y buscaremos Azure Ai Language y seleccionaremos la siguiente API.



The screenshot shows the Azure Marketplace page for the 'Language service' by Microsoft. The breadcrumb trail at the top reads 'Home > Create a resource > Marketplace'. The service is listed with a star icon and a dropdown menu. Below the service name, it says 'Microsoft | Azure Service' and '★ 3.0 (4 ratings)'. There is a badge indicating 'Azure benefit eligible'. Below this, there are two dropdown menus: 'Subscription' (set to 'Azure for Students') and 'Plan' (set to 'Language service'). A blue 'Create' button is positioned to the right of the 'Plan' dropdown. At the bottom, there are tabs for 'Overview', 'Plans', 'Usage Information + Support', and 'Ratings + Reviews'. The 'Overview' tab is selected, showing a description: 'Azure AI service for Language enables you to build apps with industry-leading natural language understanding capabilities without machine learning expertise. It consolidates familiar capabilities previously available in Language, QnA Maker, and Language Understanding (LUIS) into a unified service. With a single API call, harness decades of Microsoft's state-of-the-art research and ground-breaking NLU capabilities.'

Figura 10. Segundo servicio a utilizar.

Create Language ...

Basics Network Identity Tags Review + create

Unlock insights from unstructured text using advanced natural language processing. Use sentiment analysis to find out what customers think of your brand. Find topic-relevant phrases using key phrase extraction and identify the language of the text with language detection. Detect and categorize entities in your text with named entity recognition.

[Learn more](#)

Project Details

Subscription *	Azure for Students
Resource group *	GR-FaceRecog-API-Python

[Create new](#)

Instance Details

Region	West US 3
Name *	SentimentAnalysisA01253316
Pricing tier *	Free F0 (5K Transactions per 30 days)

[View full pricing details](#)

Responsible AI Notice

Microsoft provides technical documentation regarding the appropriate operation applicable to this Azure AI service that is made available by Microsoft. Customer acknowledges and agrees that they have reviewed this documentation and will use this service in accordance with it.

[Responsible Use of AI documentation for Text Analytics for Health](#)

Figura 11. Creación del segundo servicio.

Home

TextAnalyticsCreate-20260315221615 | Overview

Deployment

x << Delete Cancel Redeploy Download Refresh

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : TextAnalyticsCreate-20260315221615

Subscription : Azure for Students

Resource group : GR-FaceRecog-API-Python

Start time : 3/15/2026, 10:18:57 PM

Correlation ID : 10961733-b6b6-4aa6-b776-a68f888c0ef2

> Deployment details

< Next steps

Go to resource

Figura 12. Segundo recurso creado exitosamente.

Usar el servicio con un programa o aplicación

Copiaremos igual las claves y las guardaremos con nuestro manager de secretos.

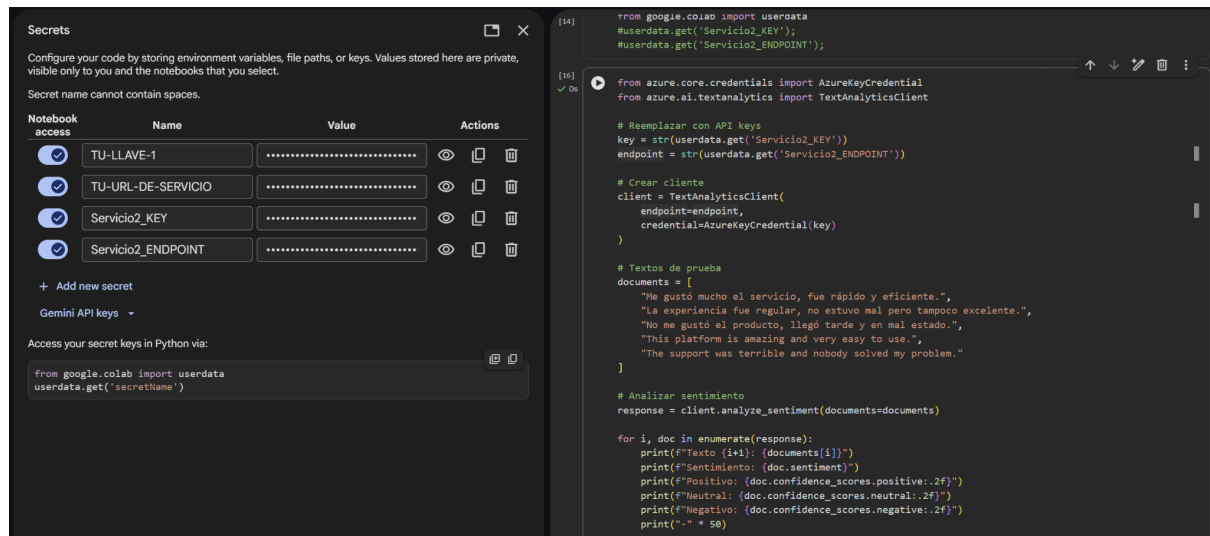


Figura 13. Llaves y código de ejemplo para análisis de sentimientos.

Creamos igual un código en donde proporcionamos el texto a analizar y conseguimos la salida de la API.

```

... Texto 1: Me gustó mucho el servicio, fue rápido y eficiente.
Sentimiento: positive
Positivo: 1.00
Neutral: 0.00
Negativo: 0.00
-----
Texto 2: La experiencia fue regular, no estuvo mal pero tampoco excelente.
Sentimiento: negative
Positivo: 0.03
Neutral: 0.08
Negativo: 0.89
-----
Texto 3: No me gustó el producto, llegó tarde y en mal estado.
Sentimiento: negative
Positivo: 0.00
Neutral: 0.00
Negativo: 1.00
-----
Texto 4: This platform is amazing and very easy to use.
Sentimiento: positive
Positivo: 1.00
Neutral: 0.00
Negativo: 0.00
-----
Texto 5: The support was terrible and nobody solved my problem.
Sentimiento: negative
Positivo: 0.00
Neutral: 0.00
Negativo: 1.00
-----

```

Figura 14. Salida del segundo servicio.

Y como podemos observar, analizar correctamente el sentimiento fue muy fácil para la API de Sentiment Analysis. Podemos ver que detectó correctamente entre cada idioma y dio un puntaje acertado sobre los fragmentos de texto.

Un uso muy común de análisis de sentimiento es hacer uso de web scraping para analizar miles de reseñas en línea, por eso haremos uso de una aplicación sencilla en donde buscaremos una reseña al azar de Letterbox y veremos que arroja la API.

```

import random
import re
import feedparser
from bs4 import BeautifulSoup
from azure.core.credentials import AzureKeyCredential
from azure.ai.textanalytics import TextAnalyticsClient

AZURE_KEY = str(userdata.get('Servicio2_KEY'))
AZURE_ENDPOINT = str(userdata.get('Servicio2_ENDPOINT'))

RSS_URL = "https://letterboxd.com/darrencb/rss/"

def clean_html(html_text):
    soup = BeautifulSoup(html_text, "html.parser")
    text = soup.get_text(" ", strip=True)
    text = re.sub(r"\s+", " ", text).strip()
    return text

def looks_like_review(text):
    # Evitar listas, rankings y contenido raro
    banned_phrases = [
        "other film categories",
        "view the full list on letterboxd",
        "plus",
        "more.",
    ]

    lower = text.lower()

    if any(phrase in lower for phrase in banned_phrases):
        return False

    # Debe tener tamaño razonable y parecer opinión
    if len(text) < 40 or len(text) > 1500:
        return False

    return True

```

Figura 15. Código para obtener reseña de Letterbox.

```

        return True

def get_random_review_from_rss(rss_url):
    feed = feedparser.parse(rss_url)
    candidates = []

    for entry in feed.entries:
        raw_text = ""

        if "summary" in entry:
            raw_text = entry.summary
        elif "description" in entry:
            raw_text = entry.description

        clean_text = clean_html(raw_text)

        if looks_like_review(clean_text):
            candidates.append(clean_text)

    if not candidates:
        return None

    return random.choice(candidates)

def analyze_sentiment(text):
    client = TextAnalyticsClient(
        endpoint=AZURE_ENDPOINT,
        credential=AzureKeyCredential(AZURE_KEY)
    )

    result = client.analyze_sentiment(documents=[text])[0]

    return {
        "sentiment": result.sentiment,
        "positive": result.confidence_scores.positive,
        "neutral": result.confidence_scores.neutral,
        "negative": result.confidence_scores.negative,
    }

```

Figura 16. Código para obtener reseña al azar y analizar sentimiento.

```

def main():
    review = get_random_review_from_rss(RSS_URL)

    if not review:
        print("No se encontró una reseña válida en el RSS.")
        return

    print("RESEÑA ALEATORIA:\n")
    print(review)
    print("\n" + "-" * 50)

    result = analyze_sentiment(review)

    print("ANÁLISIS DE SENTIMIENTO:")
    print("Sentimiento general:", result["sentiment"])
    print(f"Positivo: {result['positive']:.2f}")
    print(f"Neutral: {result['neutral']:.2f}")
    print(f"Negativo: {result['negative']:.2f}")

if __name__ == "__main__":
    main()

```

Figura 17. Código del loop principal.

Y como resultado obtenemos lo siguiente.

```

... RESEÑA ALEATORIA:

Song Sung Blue is conventional and generic in style and structure, but I found its story very likeable.

-----
ANÁLISIS DE SENTIMIENTO:
Sentimiento general: mixed
Positivo: 0.69
Neutral: 0.11
Negativo: 0.20

```

Figura 18. Análisis de sentimiento de reseña al azar de Letterbox.

Obtener los resultados y explicarlos

Corrimos varias veces el código anterior y podemos observar que analizó correctamente las reseñas en varios intentos, cosa que demuestra la gran capacidad y facilidad de hacer uso de un servicio de Análisis de sentimiento sin tener que entrenar un “Text Corpus” para poder clasificar el significado de un texto bastante amplio.

```
RESEÑA ALEATORIA:

Left-Handed Girl is a film aiming for naturalism and to capture the chaotic moments of life.

-----
ANÁLISIS DE SENTIMIENTO:
Sentimiento general: mixed
Positivo: 0.26
Neutral: 0.03
Negativo: 0.70
```

Figura 19. Primera reseña al azar.

```
RESEÑA ALEATORIA:

This review may contain spoilers. The Perfect Neighbour is a film designed to make your blood boil. It is infuriating to watch.

-----
ANÁLISIS DE SENTIMIENTO:
Sentimiento general: mixed
Positivo: 0.15
Neutral: 0.14
Negativo: 0.71
```

Figura 20. Segunda reseña al azar.

```
RESEÑA ALEATORIA:

Urchin continues the legacy of British social realist cinema, telling the story of a homeless man and the systems that fail him.

-----
ANÁLISIS DE SENTIMIENTO:
Sentimiento general: mixed
Positivo: 0.32
Neutral: 0.19
Negativo: 0.49
```

Figura 21. Tercera reseña al azar.

Como nota adicional, ya que Letterbox cuenta con una API cerrada y medidas contra Web Scraping por parte de bots, tuvimos que seleccionar un usuario al azar con buenas reseñas y hacer el análisis de sentimiento al azar en cada una de las películas que proporcionó una reseña.

Conclusión

En esta práctica se exploró el uso de APIs en plataformas de computación en la nube para integrar servicios avanzados dentro de aplicaciones. Mediante el uso de scripts en Python se comprendió cómo autenticarse, configurar credenciales y consumir servicios como traducción automática y análisis de imágenes proporcionados por proveedores como Google Cloud y Microsoft.

Estas herramientas permiten agregar capacidades inteligentes a las aplicaciones sin necesidad de desarrollar algoritmos complejos desde cero, aprovechando la infraestructura y los servicios especializados que ofrecen los proveedores de nube. De esta forma se refuerza la importancia de las APIs como un medio fundamental para conectar aplicaciones con servicios externos y ampliar sus funcionalidades de manera eficiente y escalable.

Los servicios de inteligencia artificial disponibles en la nube, como Azure AI Language, permite incorporar capacidades avanzadas de análisis de información dentro de las aplicaciones sin necesidad de desarrollar modelos complejos desde cero. De esta manera, empresas y desarrolladores pueden integrar funcionalidades avanzadas en sus sistemas de forma sencilla, mejorando la toma de decisiones basada en datos y optimizando la experiencia de los usuarios.

Bibliografía

Microsoft. (2024). Azure AI Language Documentation. Disponible en: <https://learn.microsoft.com/azure/ai-services/language-service/>

Microsoft Azure. (2024). Text Analytics / Sentiment Analysis Overview. Disponible en: <https://learn.microsoft.com/azure/ai-services/language-service/sentiment-opinion-mining/>

Python Software Foundation. (2024). Python Documentation. Disponible en: <https://docs.python.org/3/>

Letterboxd. (2024). Letterboxd Platform. Disponible en: <https://letterboxd.com/>

Microsoft. (2024). Azure SDK for Python. Disponible en: <https://learn.microsoft.com/python/api/overview/azure/ai-textanalytics-readme>